

1. Deep Learning Ensemble

A. Spatio-Temporal Transformer

The core of the Transformer is the Scaled Dot-Product Attention mechanism. It allows the model to weigh the importance of different data points across both space and time.

$$\text{Attention}(Q,K,V)=\text{softmax}(dkQKT)V$$

- Q: Query matrix
- K: Key matrix
- V: Value matrix
- dk: The dimension of the key vectors

This formula calculates a weighted sum of the values (V) based on the compatibility of the queries (Q) with the keys (K).

B. LSTM-CNN Hybrid

This architecture combines the local feature extraction of a CNN with the sequential memory of an LSTM.

- **Convolutional Layer:** The output of a 1D convolution for a sequence x is: $y(t)=(x*k)(t)=\tau\sum x(t-\tau)k(\tau)$ where k is the kernel (filter) and τ is the time delay.
- **LSTM Gates:** The LSTM cell's memory is controlled by three gates:
 - **Forget Gate:** Decides what to throw away from the previous cell state. $ft=\sigma(Wf\cdot[ht-1,xt]+bf)$
 - **Input Gate:** Decides what new information to store in the cell state. $it=\sigma(Wi\cdot[ht-1,xt]+bi)$
 - **Output Gate:** Determines the output based on the cell state. $ot=\sigma(Wo\cdot[ht-1,xt]+bo)$

C. Graph Neural Networks (GNN)

GNNs model sensor relationships by updating a node's state based on its neighbors. A common message-passing formula is:

$$h_v(k+1)=\sigma W(k)\sum_{u\in N(v)}\frac{1}{|N(v)|}h_u(k)+B(k)h_v(k)$$

- $h_v(k)$: Hidden state of node v at layer k .
- $N(v)$: Neighbors of node v .
- $\sum_{u\in N(v)}\frac{1}{|N(v)|}h_u(k)$: The aggregated message from neighbors.
- $W(k), B(k)$: Weight matrices at layer k .

D. Temporal Convolutional Networks (TCN)

TCNs use dilated causal convolutions to process long sequences efficiently. The output of a dilated causal convolution is:

$$F(s)=(x*df)(s)=\sum_{i=0}^s\sum_{k=1}^d f(i)\cdot x(s-d\cdot i)$$

- s : Current time step.

- d : Dilation factor.
- $f(i)$: A filter.
- $x(s-d \cdot i)$: The element of the input sequence at a dilated position.

E. LSTM Variational Autoencoder (VAE)

The objective function for a VAE is to minimize a combination of two terms:

$$L(x, \hat{x}) = \text{Reconstruction Loss}(x, \hat{x}) + \beta \cdot \text{KL Divergence}(q\phi(z|x) || p\theta(z))$$

- **Reconstruction Loss**: Measures how well the VAE reconstructs the input data (e.g., Mean Squared Error).
- **KL Divergence**: Regularizes the latent space by ensuring the learned distribution $q\phi(z|x)$ is close to a prior distribution $p\theta(z)$, usually a standard Gaussian.

2. Time Series Specialists

A. Prophet/NeuralProphet

Prophet models time series data using a decomposable additive model:

$$y(t) = g(t) + s(t) + h(t) + \epsilon t$$

- $g(t)$: Trend component (non-periodic changes).
- $s(t)$: Seasonality component (e.g., daily, weekly, yearly).
- $h(t)$: Holiday component (regular, country-specific events).
- ϵt : Error term.

B. ARIMA-GARCH

This model combines an ARIMA model for mean time series dynamics with a GARCH model for volatility.

- **ARIMA(p,d,q)**: $(1 - \sum_{i=1}^p \phi_i L)(1 - L)^d y_t = (1 + \sum_{i=1}^q \theta_i L) \epsilon_t$
 - L : Lag operator.
 - p, d, q : Orders of the autoregressive, differencing, and moving average terms.
- **GARCH(p,q)**: Models the conditional variance σ_t^2 of the error term: $\sigma_t^2 = \omega + \sum_{i=1}^p \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^q \beta_j \sigma_{t-j}^2$
This formula shows that the current volatility depends on past squared errors (ϵ_{t-i}^2) and past volatilities (σ_{t-j}^2).

C. Kalman Filters

A Kalman Filter operates in two steps: prediction and update.

- **Prediction Step**:
 - State prediction: $\hat{x}^k | k-1 = F_k \hat{x}^{k-1} | k-1 + B_k u^k$
 - Covariance prediction: $P_k | k-1 = F_k P_{k-1} | k-1 + F_k T + Q_k$
- **Update Step**:
 - Kalman Gain: $K_k = P_k | k-1 H_k^T (H_k P_k | k-1 H_k^T + R_k)^{-1}$
 - State update: $\hat{x}^k | k = \hat{x}^k | k-1 + K_k (z_k - H_k \hat{x}^k | k-1)$

- Covariance update: $P_{k+1} = (I - K_k H_k) P_k$

D. Wavelet Transform Analysis

The Continuous Wavelet Transform (CWT) analyzes a signal at different scales and positions.

$$\text{CWT}(a, \tau) = a^{-1/2} \int_{-\infty}^{\infty} f(t) \psi^*(at - \tau) dt$$

- a : Scale parameter (related to frequency).
- τ : Shift parameter (related to time).
- $\psi(t)$: The mother wavelet function.

3. Gradient Boosting Ensemble

All gradient boosting models iteratively build a strong learner from a series of weak learners. The objective function to be minimized is:

$$L(y^*(t)) = \sum_{i=1}^n l(y_i, y^*(t)) + \sum_{k=1}^K \Omega(f_k)$$

- $y^*(t)$: The prediction at iteration t .
- $l(y_i, y^*(t))$: The training loss.
- $\Omega(f_k)$: The regularization term for the new weak learner f_k .
- **The models differ in how they optimize this function and handle data.** For example, XGBoost uses a second-order Taylor expansion to approximate the loss.

4. Anomaly Detection Specialists

A. Isolation Forest

This algorithm works by calculating the average path length for an observation on a forest of random trees. A shorter path length means it's an anomaly.

$$h(x) = E(\text{path_length}(x, T))$$

- $h(x)$: Anomaly score for observation x .
- $\text{path_length}(x, T)$: The path length of x in a specific tree T .

B. One-Class SVM

This model finds a hyperplane that separates the data from the origin, with a focus on capturing the boundary of the normal data.

$$\min_{w, \rho, \xi} \frac{1}{2} \|w\|^2 + \nu \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad w \cdot x_i + \rho \geq 1 - \xi_i$$

- w : The normal vector to the hyperplane.
- ρ : The offset (boundary).
- ξ_i : Slack variables for each data point (penalizing points inside the boundary).
- ν : A parameter controlling the trade-off between the margin and the number of training errors.

C. Autoencoders

An Autoencoder's primary goal is to minimize the reconstruction error. For fire detection, this

is a measure of how well the model can rebuild the input sensor data.

$$\text{Loss} = \frac{1}{n} \sum (x_i - \hat{x}_i)^2$$

- x_i : The original input data.
- $\hat{x}_i$: The reconstructed output.
- **Large reconstruction errors indicate an anomaly.**

D. LSTM-VAE

This is covered in the Deep Learning section above.

5. Meta-Learning & Ensemble

A. Stacking Ensemble

This is a two-level architecture. The Level 1 models make predictions, and a Level 2 model (the meta-learner) learns to combine these predictions.

$$\hat{y} = f_{\text{meta}}(\hat{y}^1, \hat{y}^2, \dots, \hat{y}^n)$$

- $\hat{y}^i$: Prediction of the i -th Level 1 model.
- f_{meta} : The Level 2 meta-learner function.

B. Bayesian Model Averaging

The final prediction is a weighted average of the predictions from individual models, where the weights are the posterior probabilities of the models.

$$p(y | D) = \sum_i p(y | M_i, D) p(M_i | D)$$

- $p(y | D)$: The final prediction distribution.
- $p(y | M_i, D)$: Prediction from model M_i .
- $p(M_i | D)$: The posterior probability of model M_i given the data.

C. Dynamic Ensemble Selection

This is a sophisticated form of stacking where the meta-learner selects or weights the models based on the characteristics of the *input data point* itself. The formula is a weighted sum where the weights are a function of the input:

$$\hat{y} = \sum_i w_i(x) \hat{y}_i$$

- $w_i(x)$: The weight for model i , which depends on the input feature vector x .